Identity & Access Management (IAM) Web Application

Technical Report

- **Author:** Sri Hari Aravindan

- **Stack:** Python 3.10 · Flask · SQLite · Bootstrap 5 · Werkzeug
- **Deployment:** Replit / Render (free tier)
- **Repository:** github.com/Panca2022/iam-project

---

1. Abstract

This project is a full-stack Identity and Access Management (IAM) web application built on Python Flask with an SQLite backend. It demonstrates the core pillars of IAM - authentication, authorization, session management, auditing, and self-service - in a compact, deployable codebase. The system implements Role-Based Access Control (RBAC) with two roles (admin, user), enforces a password strength policy, mitigates brute-force login attempts through account lockout, records every security-relevant event into an audit log, and hardens session cookies against XSS/CSRF. It is deliberately minimalist so each design decision can be defended in an interview.

2. Objectives

- Demonstrate Authentication (verify identity) and Authorization (control access).
- Enforce least privilege - users only see what their role allows.
- Preserve an audit trail for every identity event.
- Practice secure development habits (hashing, session hygiene, rate limiting).
- Remain deployable on free infrastructure (Replit / Render).

3. Architecture

```
Browser  ──HTTP──▶  Flask app (app.py)  ──SQL──▶  SQLite (database.db)
                          │
                          ├── templates/*.html   (Jinja2 views)
                          ├── decorators          (RBAC)
                          └── audit_logs table    (immutable event store)
```

- **Presentation layer** - Jinja2 templates + Bootstrap for a responsive UI.
- **Application layer** - Flask routes; each route is protected by a decorator that enforces authentication or role membership.
- **Data layer** - SQLite with parameterized queries (no string concatenation → no SQL injection).
- **Cross-cutting concerns** - Audit logging and password policy are implemented as helper functions called from routes.

3.1 Project Structure

```
IAM_Project/
│
├─ app.py                    # Main Flask application (routes + RBAC + audit)
├─ init_db.py                # Database schema + seed admin user
├─ requirements.txt          # Python dependencies
├─ database.db               # SQLite database (generated)
├─ render.yaml               # Render.com deployment manifest
├─ templates/
│  ├─ home.html
│  ├─ login.html
│  ├─ register.html
│  ├─ admin.html             # Admin dashboard (users + audit log)
│  ├─ user.html              # User dashboard
│  ├─ change_password.html   # Self-service password change
│  ├─ logout.html
│  └─ 403.html               # Forbidden error page
└─ static/
   └─ style.css
```

4. Component Breakdown

Component	Responsibility
`register()`	Validates password strength, hashes with Werkzeug PBKDF2, inserts user.
`login()`	Verifies hash, checks lockout, resets failed counter, sets session.
`admin_page()`	Lists users with pagination + search; reads audit log.
`delete_user()`	Soft delete - flips `is_active` so history is preserved.
`change_password()`	Self-service re-credentialing with policy enforcement.
`log_action()`	Centralized writer to `audit_logs`.
`@login_required` / `@admin_required`	Declarative RBAC - permission lives with the route.

5. Security Model

Control	Threat Mitigated	Why It Matters
Werkzeug PBKDF2 hashing	Credential theft from DB dump	Passwords are never recoverable, even if the DB leaks.
Password strength policy	Weak-credential brute force	Aligns with NIST SP 800-63B and OWASP ASVS §2.1.
Failed-attempt lockout	Online brute force	5 attempts / 15 min is industry default (CIS, NIST).

Control	Threat Mitigated	Why It Matters
HttpOnly + SameSite cookies	XSS cookie theft, CSRF	Reduces impact of two of the OWASP Top 10.
30-minute idle timeout	Session hijacking window	Shrinks attack window after device theft.
Parameterized SQL	SQL injection	Prevents the #1 classic web exploit.
Audit log	Repudiation	Supports forensics and compliance (SOC2, ISO 27001 A.12.4).
Soft delete	Data loss / loss of accountability	Hard-deleting users destroys history; deactivating preserves it.
`@admin_required` decorator	Vertical privilege escalation	Authorization checked at route boundary - no path can bypass it.

6. Why This Architecture

- **Flask over Django** - For an IAM demo, showing explicit auth logic (hash check, session set, decorator) is more convincing than hiding it inside Django's auth package. The interviewer can see the mechanics.
- **SQLite over PostgreSQL** - Zero-config, file-based, perfect for a portable demo. The SQL is standard and would port to Postgres with a single connection-string change.
- **Server-rendered templates over SPA** - Session cookies work naturally; no token-in-localStorage XSS surface; simpler to reason about security.
- **Decorators over inline if checks** - Permission logic is co-located with the route, easier to audit and impossible to forget on a new route.
- **Centralized `log_action()`** - A single writer guarantees consistent log format; a new route only needs one call to be auditable.

7. Database Schema

7.1 `users`

Column	Type	Purpose
`id`	INTEGER PK	Surrogate key
`username`	TEXT UNIQUE	Login identifier
`password`	TEXT	PBKDF2 hash
`role`	TEXT	admin or user
`failed_attempts`	INTEGER	Lockout counter
`locked_until`	TEXT (ISO)	Lockout expiry
`last_login`	TEXT (ISO)	For audit / admin display
`is_active`	INTEGER	Soft-delete flag
`created_at`	TEXT	Auto

7.2 audit_logs

Column	Type	Purpose
id	INTEGER PK	Auto
user_id	INTEGER	FK to users (nullable)
username	TEXT	Snapshot of username at event time
action	TEXT	e.g. LOGIN_SUCCESS, USER_DEACTIVATED
target	TEXT	Object of the action
ip_address	TEXT	Request origin
timestamp	TEXT	Auto

8. Password Policy

Enforced in validate_password_strength():

- Minimum length: 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (!@#$%^&*(),.?":{}|<>_-+=)

Enforced server-side - the browser's minlength attribute is UX only and can be bypassed with curl, DevTools, or Postman.

9. Testing Evidence

9.1 Server-Side Rejection of Weak Password (proven via curl)

```
curl -X POST http://127.0.0.1:3000/register ^
     -d "username=hacker1" ^
     -d "password=abc" ^
     -d "role=user"
```

Server response contains:

```
<div class="alert alert-danger">Password must be at least 8 characters.</div>
```

Database check confirms no insertion:

```
python -c "import sqlite3; c=sqlite3.connect('database.db');
print(c.execute(\"SELECT * FROM users WHERE username='hacker1'\").fetchall())"
```

Output:

```
[]
```

9.2 Test Matrix

#	Username	Password	Expected
1	testuser1	`Ab@1`	❌ "at least 8 characters"
2	testuser2	`hello@1234`	❌ "one uppercase letter"
3	testuser3	`HELLO@1234`	❌ "one lowercase letter"
4	testuser4	`Hello@World`	❌ "one digit"
5	testuser5	`Hello12345`	❌ "one special character"
6	testuser6	`Hello@1234`	✅ Registered, redirect to /login
7	hacker1 (curl)	`abc`	❌ Rejected despite no browser

9.3 Lockout Test

1. Attempt login with wrong password 5 times.
2. 6th attempt → "Account temporarily locked."
3. Wait 15 min (or update `locked_until` in DB) → login works again.

9.4 Soft-Delete Test

1. Admin clicks Deactivate on a user row.
2. Row's status badge flips to Inactive.
3. `audit_logs` records `USER_DEACTIVATED`.
4. Deactivated user cannot log in (`is_active` check in `login()`).

10. Future Roadmap

Phase	Item	Rationale
1	Flask-WTF CSRF tokens	Defense-in-depth beyond SameSite cookies.
2	Email verification on register	Prevents fake-account registration.
3	TOTP-based MFA	The single biggest security win for IAM.
4	Password reset via signed token	Standard self-service flow.

Phase	Item	Rationale
5	Postgres + Alembic migrations	Production-grade persistence.
6	OAuth2 / OIDC login (Google, Entra)	Federated identity - the modern IAM pattern.
7	Role → Permission mapping table	Move from 2 roles to granular permissions (ABAC-ready).

11. Observations

- The codebase uses `generate_password_hash` from Werkzeug - a strong sign of credential storage awareness, which is the #1 IAM concept.
- Authorization was originally checked inline in each route (`if session['role'] != 'admin'`). Consolidating into `@admin_required` is a code-quality story: "I refactored scattered checks into a single decorator so every protected route declares its requirement."
- The audit log is what turns this from a class assignment into something that sounds like a real IAM product. It allows the candidate to say: "Every login, failed login, logout, password change, and user deactivation is logged with user, target, IP, and timestamp."

12. Summary - 30-Second Elevator Pitch

> "It's a Flask-based IAM demo. Users register with strong passwords that get PBKDF2-hashed. Login enforces a 5-attempt lockout and sets a hardened session cookie. Routes are protected by `@login_required` and `@admin_required` decorators so authorization is declarative. Admins see all users with pagination and search, and can deactivate - not delete - accounts so the audit log stays intact. Every identity event is written to an `audit_logs` table with user, target, IP, and timestamp. I deliberately chose Flask over Django so the auth mechanics are visible, and SQLite so the whole thing runs with `python app.py`."

13. Deployment Notes

- **Local:** `python init_db.py && python app.py` → `http://127.0.0.1:3000`
- **Render:** `render.yaml` uses `gunicorn app:app`. Set `SECRET_KEY` and `FLASK_ENV=production` as environment variables.
- **Database migration to Postgres:** replace `sqlite3.connect` with a Postgres driver; SQL is otherwise standard.

14. Glossary

- **IAM** - Identity and Access Management.
- **RBAC** - Role-Based Access Control.
- **PBKDF2** - Password-Based Key Derivation Function 2 (used by Werkzeug hashing).
- **CSRF** - Cross-Site Request Forgery.
- **XSS** - Cross-Site Scripting.
- **Soft delete** - Marking a row inactive instead of physically removing it.
- **Audit log** - Append-only record of security-relevant events.
- **Session hardening** - Cookie flags (HttpOnly, SameSite, Secure) + timeout to limit session attacks.
